QATIP Intermediate AWS Lab 03
Terraform authentication in VS Code

Windows and PowerShell 7.4 or later

Lab objectives 1
Before you begin 1
Select and verify your starting identity 2
Retrieve credentials and run Terraform 3
Restore the starting identity and create a role 4
Configure Terraform to assume the role 5
Demonstrate a change and complete cleanup 6
Troubleshooting and references 8

Lab objectives

Use a named AWS profile as your starting identity, retrieve a demonstration IAM user's credentials from AWS Secrets Manager, and then configure Terraform to assume an IAM role. Verify the identity at each transition and remove the lab-created resources at the end.

You perform both administrator and developer tasks for the exercise. Secrets Manager stores a credential; it does not remove the need to authenticate before retrieving that credential. The IAM user keys remain long-lived even when stored in Secrets Manager. The role exercise shows the use of temporary credentials through AWS STS.

Before you begin

1. In VS Code, select File then Open Folder and open your local aws-tf-int repository. In Explorer, right-click labs/03 and choose Open in Integrated Terminal.

```
$PSVersionTable.PSVersion
Get-Location
Get-Command aws, terraform
Get-ChildItem
```

Confirm PowerShell is version 7.4 or later and the current directory ends in labs\03. You should see main.tf, its Terraform support files and the five PowerShell helpers. Use this same terminal throughout the lab.

Select and verify your starting identity

2. List your saved AWS profiles. Set the profile and region for this lab; replace default if your course uses another profile name.

```
aws configure list-profiles
$LabProfile = 'default'
$LabRegion = 'us-west-2'
```

3. Confirm that this profile works before changing the terminal credentials.

```
$LabIdentityJson = aws sts get-caller-identity --profile $LabProfile --output json
if ($LASTEXITCODE -ne 0) { throw "Lab profile authentication failed" }
$LabIdentity = $LabIdentityJson | ConvertFrom-Json
$LabIdentity | Select-Object Account, Arn
```

4. Remove session credential overrides, then select the saved lab profile explicitly.

```
Remove-Item Env:AWS_ACCESS_KEY_ID -ErrorAction SilentlyContinue
Remove-Item Env:AWS_SECRET_ACCESS_KEY -ErrorAction SilentlyContinue
Remove-Item Env:AWS_SESSION_TOKEN -ErrorAction SilentlyContinue
Remove-Item Env:AWS_SECURITY_TOKEN -ErrorAction SilentlyContinue
Remove-Item Env:AWS_DEFAULT_PROFILE -ErrorAction SilentlyContinue
Remove-Item Env:AWS_ROLE_ARN -ErrorAction SilentlyContinue
Remove-Item Env:AWS_WEB_IDENTITY_TOKEN_FILE -ErrorAction SilentlyContinue
$env:AWS_PROFILE = $LabProfile
$env:AWS_REGION = $LabRegion
$env:AWS_DEFAULT_REGION = $LabRegion
$env:AWS_PAGER = ""
aws sts get-caller-identity
```

The account and ARN should match the saved LabIdentity values. This is the starting identity you will return to later; it need not have the literal user name student.

Administrator creates the demonstration user and secret

5. Read Setup-TerraformSecrets.ps1 in VS Code, then run it.

```
Get-ChildItem -File -Filter *.ps1 | Unblock-File
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned -Force
.\Setup-TerraformSecrets.ps1 -Profile $LabProfile -Region $LabRegion
```

The helper checks the supplied policy, creates terraform-user, attaches terraform-user-policy, creates one access key, and stores the key pair in terraform-aws-credentials in the selected region. It stops if the named user or secret already exists, rather than overwriting it. Keep lab03-vscode-resources.json; it records what the helpers created for cleanup.

6. Verify the secret metadata without displaying its credential values.

```
aws secretsmanager describe-secret --secret-id terraform-aws-credentials
```

Retrieve credentials and run Terraform

7. Read Use-TerraformCredentials.ps1, then run it in the same PowerShell terminal.

```
.\Use-TerraformCredentials.ps1 -Profile $LabProfile -Region $LabRegion
aws sts get-caller-identity
```

The helper authenticates through your starting profile to retrieve the secret, sets the credential environment variables in this PowerShell process, and removes the active profile variables. The ARN should now identify terraform-user. Do not add --profile to this verification command: that would test a named profile instead of the active terminal credentials.

No aws_credentials.env file is created or loaded. Do not display secret values or run the helper in a separate pwsh process: Terraform must inherit the credentials from the same terminal session.

Review the Terraform configuration

8. Open main.tf. Update the S3 bucket name to a globally unique name. Find the bucket argument by its resource block rather than relying on a line number.

For this first deployment, leave the supplied assume_role block disabled. The AWS provider must not contain explicit access_key, secret_key, token or profile settings that override the terminal credentials. Retain the course-required region, tags and other resource settings. Keep credentials out of Terraform files and tfvars.

9. Initialise, validate and review the deployment plan.

```
terraform init
terraform validate
terraform plan
```

Expect the lab's S3 bucket to be proposed for creation. If identity, permissions or region are incorrect, resolve the issue before applying.

10. Apply the configuration and confirm the result.

```
terraform apply
```

Review the proposal and enter yes when it matches the intended lab deployment. Check the bucket in the AWS Console for the same account and region.

Teaching point

The identity retrieving a secret needs permission to read that secret. The stored IAM user needs permission to manage the Terraform resources. These are different responsibilities. In production, a developer reader would be granted only the necessary retrieval permissions; using the same broadly authorised lab identity to simulate both jobs does not prove that read-only access has been enforced.

Restore the starting identity and create a role

11. Clear the retrieved user keys and reselect the saved lab profile.

```
Remove-Item Env:AWS_ACCESS_KEY_ID -ErrorAction SilentlyContinue
Remove-Item Env:AWS_SECRET_ACCESS_KEY -ErrorAction SilentlyContinue
Remove-Item Env:AWS_SESSION_TOKEN -ErrorAction SilentlyContinue
$env:AWS_PROFILE = $LabProfile
aws sts get-caller-identity
```

Compare the account and ARN with LabIdentity from step 5. Clearing keys alone does not guarantee a return to the right identity on a local workstation; explicitly selecting and checking the profile makes the transition visible.

Create the IAM role

12. Read New-TerraformRole.ps1, then run it using the starting profile.

```
.\New-TerraformRole.ps1 -Profile $LabProfile
```

The helper creates TerraformRole and attaches the existing terraform-user-policy. It generates trust-policy.vscode.json for the current lab principal. If the starting identity is an assumed role, the helper resolves its underlying IAM role ARN instead of trusting a single STS session ARN. It stops if TerraformRole already exists.

The trust relationship identifies who may assume the role. The attached permissions policy identifies what the role may do. Your starting identity must also have any required sts:AssumeRole permission; the helper does not change permissions on an existing source identity.

13. Inspect the generated trust document and the role returned by AWS.

```
Get-Content .\trust-policy.vscode.json
aws iam get-role --role-name TerraformRole --no-cli-pager
aws iam list-attached-role-policies --role-name TerraformRole --no-cli-pager
```

Locate AssumeRolePolicyDocument and confirm the intended principal. Confirm terraform-user-policy is attached. The trust policy itself does not grant S3 permissions.

14. Read the current default version of the permissions policy.

```
$LabResources = Get-Content .\lab03-vscode-resources.json -Raw | ConvertFrom-Json
$PolicyArn = $LabResources.PolicyArn
$PolicyJson = aws iam get-policy --policy-arn $PolicyArn --output json
$Version = ($PolicyJson | ConvertFrom-Json).Policy.DefaultVersionId
aws iam get-policy-version --policy-arn $PolicyArn --version-id $Version
```

Review the permitted actions and resources. The policy's default version may not be v1; the commands above find the active version.

Configure Terraform to assume the role

15. Print the role ARN from the lab record, then copy it for the next edit.

```
$LabResources.RoleArn
```

16. In the existing default AWS provider block in main.tf, enable the assume_role block shown below. Update {change-me} with the value just printed. Preserve other required provider settings.

```
assume_role {
  role_arn = "arn:aws:iam::{change-me}:role/TerraformRole"
  session_name = "qatipint-lab03"
}
```

17. Enable the supplied identity data source and output. If the lab files do not already define them, add the following once; do not duplicate existing blocks.

```
data "aws_caller_identity" "current" {}

output "terraform_identity" {
  value = data.aws_caller_identity.current.arn
}
```

If the supplied data source uses a different local name, use that name in the output reference.

18. Initialise, validate and review the role-based plan.

```
terraform init
terraform validate
terraform plan
```

The bucket already exists, so changing authentication alone should not require resource creation or deletion. The terraform_identity output may change. IAM changes can take a short time to propagate; retry the plan after a brief wait if the new role is not yet available. An AccessDenied error that persists needs the trust policy, source permission and role policy checked.

19. Apply the plan and verify both identities.

```
terraform apply --auto-approve
terraform output terraform_identity
aws sts get-caller-identity
```

The Terraform output should contain arn:aws:sts::<ACCOUNT_ID>:assumed-role/TerraformRole/qatipint-lab03. The separate AWS CLI command should still show the starting lab identity. The provider's assume_role setting changes the identity used by Terraform, not the identity of the PowerShell terminal or every AWS CLI command.

Demonstrate a change and complete cleanup

20. Change the lab bucket name to another unique name, then run terraform plan.

```
terraform plan
```

The plan should show replacement of the lab bucket. Use only the empty demonstration bucket created in this lab. Review the replacement and apply it, then confirm that terraform_identity still identifies TerraformRole.

```
terraform apply --auto-approve
terraform output terraform_identity
```

Destroy Terraform resources before removing authentication

21. Leave assume_role configured and the starting profile selected. Destroy the resources while TerraformRole still exists.

    ```
    terraform destroy
    ```

    Review the destroy plan and confirm it. Verify Terraform reports that the lab resources have been destroyed before continuing. If deletion fails, resolve it before removing the role or access key.

22. Confirm the AWS CLI is using the starting lab profile, then run the cleanup helper.

    ```
    aws sts get-caller-identity
    .\Remove-LabAuthentication.ps1 -Profile $LabProfile
    ```

    Note: In the class lab, this phase may fail due to restricted permission for aws/student, namely iam:DetachRolePolicy. If an error occurs then continue to the next step.

    The helper checks the account against the lab record, then removes the recorded IAM role, access key, IAM user and lab secret. The demonstration secret is deleted without a recovery window. The pre-existing terraform-user-policy is retained. Keep the lab record until cleanup succeeds. A just-deleted secret name may take a short time to become reusable.

23. Leave the terminal on the starting profile and confirm it one final time.

    ```
    $env:AWS_PROFILE = $LabProfile
    aws sts get-caller-identity
    ```

What you have demonstrated

You selected a local AWS identity explicitly, retrieved a separate IAM user's credentials, deployed Terraform resources with that user, and then used a role's temporary credentials through the AWS provider. You also distinguished terminal identity from provider identity and cleaned up both infrastructure and authentication resources.

Troubleshooting and references

The terminal rejects a PowerShell script

Confirm the terminal is PowerShell 7.4 or later and that all five helper files are together. Review downloaded scripts before unblocking them. On a training desktop that permits local scripts, the commands below unblock the lab scripts and select RemoteSigned for this terminal process only. A Group Policy restriction still takes precedence; ask the trainer if it prevents execution.

```
Get-ChildItem -File -Filter *.ps1 | Unblock-File
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

No named profile or an expired session

Run aws configure list-profiles and check the profile name. Complete the AWS sign-in or temporary credential setup supplied for your lab. A profile using temporary credentials needs its session token. Check with aws sts get-caller-identity --profile <PROFILE> before repeating the lab setup.

The wrong identity is active

Run aws sts get-caller-identity without --profile to inspect the active terminal identity. Check for session access-key variables, profile settings and explicit credentials in the provider. Return to steps 6 or 13 as appropriate. A new terminal does not inherit credential changes made in a different terminal.

A helper reports an existing resource or a missing policy

The helpers do not adopt or overwrite existing named IAM resources. Ask the trainer to confirm their ownership before removing anything. The platform administrator must provide terraform-user-policy. The helpers require this policy before creating the demonstration user or role. If setup partly failed, use its retained lab record to complete cleanup.

Terraform cannot assume the new role

Check the role trust principal, the starting identity's sts:AssumeRole permission and any permissions boundaries or organisation controls. Confirm the role ARN copied into main.tf matches the lab record. Avoid upgrading providers merely to resolve an authentication change.

Further reading

AWS credential environment variables
AWS Secrets Manager retrieval
AWS IAM temporary credential guidance
AWS provider assume role guidance

© QA 2026

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026